



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2024/2025

SECP 2753 – DATA MINING

SECTION 01

PROJECT 2 - UNSUPERVISED LEARNING

LECTURER: DR. ROLIANA BINTI IBRAHIM

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156

Table of Contents

1.0 INTRODUCTION	2
2.0 DATASET - No-Show Appointments	3
2.1 Data and Business Understanding	3
2.2 Data Preparation and Preprocessing	4
2.3 Model Development (Unsupervised Learning)	7
2.4 Model Performance & Evaluation (Elbow Method, Silhouette Score)	10
3.0 CONCLUSION	11

1.0 INTRODUCTION

In the contemporary healthcare landscape, scheduled appointments offer significant convenience for patients and optimize resource management within clinics and hospitals. However, a persistent challenge for healthcare providers is the issue of missed medical appointments, commonly known as "no-shows."

While some may view this as a minor inconvenience, it can lead to substantial waste of resources, impacting resource planning, staff efficiency, and potentially delaying care for other patients. To mitigate these negative effects, **unsupervised learning** offers a powerful approach. Instead of directly predicting individual no-shows, unsupervised methods can help in **discovering hidden patterns, segments, or characteristics within appointment data**. By clustering patients based on their historical behavior or identifying the underlying factors contributing to no-show patterns, healthcare systems can gain deeper insights, inform strategic interventions, and optimize resource allocation more effectively.

2.0 DATASET - No-Show Appointments

For this report, we are going to analyze the second dataset regarding patients who missed their medical appointments as well as running a machine learning model to predict whether the patient will likely show up to their medical appointment or not and evaluate its performance. The original dataset can be obtained from this link: [📄 noshowappointments](#)

2.1 Data and Business Understanding

Problem Statement:

To predict whether a patient will likely show up for their scheduled medical appointment.

Overview:

In this part, we will focus on understanding the business problem and exploring the dataset where the main goal is to predict whether a patient will likely show up for their scheduled medical appointment, to improve the management in the healthcare ecosystem. Firstly, we will explore the dataset structure, types of features, and initial data quality. Basic analysis helps identify key characteristics such as missing values, imbalanced classes, and anomalies (e.g., negative age). These insights guide the data cleaning and feature engineering steps that follow.

Importance:

- By identifying a patient that has high risk of missing their medical appointment, an alternative action can be taken by reminding them of their medical appointment and rescheduling it if any.
- Predicting a potential no-shows trends can help healthcare providers in managing the schedule by rescheduled the unused appointment to reduce downtime

2.2 Data Preparation and Preprocessing

For unsupervised learning, clustering, data preparation and preprocessing are equal. It is more important than supervised learning. This phase ensures that the input data is optimized for the clustering algorithm to identify the patterns and groupings without the guidance of predefined labels. It involves meticulous cleaning, feature engineering and most importantly the appropriate scaling of features.

The data preprocessing for Project 2 adapted some of the methods from Project 1, with key modifications tailored to unsupervised learning:

```
# 1. Initial Cleaning: Handle anomaly in Age
# Remove rows where Age is less than 0
initial_rows = df.shape[0]
df = df[df['Age'] >= 0]
rows_removed = initial_rows - df.shape[0]
print("Removed {} rows with invalid Age (Age < 0). Remaining rows: {}".format(rows_removed, df.shape[0]))
```

Removed 1 rows with invalid Age (Age < 0). Remaining rows: 110526

1. Handling Anomalies (Age = -1): The record containing an invalid Age value of -1 was removed to maintain the data integrity.

```
# 2. Feature Engineering from Date/Time Columns
df['ScheduledDay'] = pd.to_datetime(df['ScheduledDay'])
df['AppointmentDay'] = pd.to_datetime(df['AppointmentDay'])

# Ensure 'AppointmentDay' has no time component for accurate day difference calculation
df['AppointmentDay'] = df['AppointmentDay'].dt.normalize()

# Calculate 'WaitingDays': Difference between AppointmentDay and ScheduledDay
df['WaitingDays'] = (df['AppointmentDay'] - df['ScheduledDay']).dt.days
df.loc[df['WaitingDays'] < 0, 'WaitingDays'] = 0 # Cap negative waiting days at 0
print("Engineered 'WaitingDays' feature.")

# Extract Day of Week and Month
df['AppointmentDayOfWeek'] = df['AppointmentDay'].dt.dayofweek # Monday=0, Sunday=6
df['ScheduledDayOfWeek'] = df['ScheduledDay'].dt.dayofweek # Monday=0, Sunday=6
df['MonthOfAppointment'] = df['AppointmentDay'].dt.month
print("Engineered 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment'.")
```

Engineered 'WaitingDays' feature.
Engineered 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment'.

2. Feature Engineering: ScheduledDay and AppointmentDay columns were converted to datetime objects. The WaitingDays feature was engineered, calculating the duration between scheduling and appointment, with negative values capped at 0. Temporal features like AppointmentDayOfWeek (the day of the week on which the appointment is scheduled), ScheduledDayOfWeek (the day of the week when the appointment was scheduled) and MonthOfAppointment (the month in which the appointment falls) were extracted to capture cyclical patterns.

```

# 3. Categorical Feature Encoding
# Encode 'Gender': 'F' -> 0, 'M' -> 1
df['Gender'] = df['Gender'].map({'F': 0, 'M': 1})
print("Encoded 'Gender' column (F=0, M=1).")

# Prepare 'No-show' column but note it will be dropped later for unsupervised learning
# (Applying the same cleaning as in Project 1 to ensure data consistency if it were ever used)
df['No-show'] = df['No-show'].str.strip().str.capitalize()
df['No-show'] = df['No-show'].map({'No': 0, 'Yes': 1})
# Fill NaNs that might result from mapping (e.g., if other values existed)
if df['No-show'].isnull().any():
    print("WARNING: NaNs found in 'No-show' after mapping. Filling with 0. NaNs before fill: {}".format(df['No-show'].isnull().sum()))
    df['No-show'].fillna(0, inplace=True)
else:
    print("'No-show' column mapped successfully with no NaNs introduced (will be dropped).")

# Apply One-Hot Encoding to 'Neighbourhood'
# This handles the many unique values of Neighbourhood.
df_processed = pd.get_dummies(df, columns=['Neighbourhood'], prefix='Neighbourhood')
print("One-Hot Encoded 'Neighbourhood' column.")

Encoded 'Gender' column (F=0, M=1).
'No-show' column mapped successfully with no NaNs introduced (will be dropped).
One-Hot Encoded 'Neighbourhood' column.

```

3. Categorical Feature Encoding: The Gender column was numerically encoded (F to 0, M to 1). The Neighbourhood column, being a nominal categorical variable with numerous unique entries, was transformed using One-Hot Encoding. This process expands the single categorical column into multiple binary columns to prevent the algorithm from unintended ordinal relationship.

```

# 4. Feature Removal for Unsupervised Learning
# Drop PatientID, AppointmentID, original date columns (ScheduledDay, AppointmentDay)
# CRUCIAL: Also drop the 'No-show' column as it is the target in supervised learning,
# but for unsupervised clustering, we are looking for patterns in the features themselves.
columns_to_drop = ['PatientID', 'AppointmentID', 'ScheduledDay', 'AppointmentDay', 'No-show']
df_final_features = df_processed.drop(columns=columns_to_drop, axis=1)
print("Dropped identifier columns, original date columns, and the 'No-show' target variable.")

Dropped identifier columns, original date columns, and the 'No-show' target variable.

```

4. Feature Removal: Unique identifiers (PatientID, AppointmentID) and the original date columns were removed. In unsupervised learning, the goal is to discover inherent structures within the data itself, without relying on a predefined outcome.

```

# 5. Feature Scaling (CRUCIAL for K-Means Clustering)

numerical_cols_to_scale = ['Age', 'WaitingDays', 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment']

# Ensure only columns that exist are included in numerical_cols_to_scale
numerical_cols_to_scale = [col for col in numerical_cols_to_scale if col in df_final_features.columns]

if numerical_cols_to_scale:
    scaler = StandardScaler()
    df_final_features[numerical_cols_to_scale] = scaler.fit_transform(df_final_features[numerical_cols_to_scale])
    print("Applied StandardScaler to numerical features: {}".format(', '.join(numerical_cols_to_scale)))
else:
    print("No numerical columns found for scaling.")

```

5. Feature Scaling (Standardization): This step is very important for K-Means clustering. K-Means works by measuring distance between data points, so if one feature (like Age or WaitingDays) has much bigger numbers than others, it can affect the results too much and make the other features less important.

To fix this, we used StandardScaler on the numerical features. This method changes the values so that each feature has an average of 0 and the same spread, which is standard deviation of 1.

By doing this, all the features are on the same scale, so they each play an equal role in the clustering. This avoids one feature from having too much influence just because of its larger values.

	Gender	Age	Scholarship	Hipertension	Diabetes	Alcoholism	Handcap	SMS_received	WaitingDays	AppointmentDayOfWeek	...	Neighbourhood_SANTOS REIS	N
0	0	1.077926	0	1	0	0	0	0	-0.63435	1.561421	...	False	
1	1	0.818297	0	0	0	0	0	0	-0.63435	1.561421	...	False	
2	0	1.077926	0	0	0	0	0	0	-0.63435	1.561421	...	False	
3	0	-1.258733	0	0	0	0	0	0	-0.63435	1.561421	...	False	
4	0	0.818297	0	1	1	0	0	0	-0.63435	1.561421	...	False	

The preprocessing pipeline successfully transformed the raw data into a suitable format for unsupervised learning. The Age anomaly was handled, and various temporal features were engineered. Categorical variables like Gender and Neighbourhood were appropriately encoded. All numerical features identified for scaling have been standardized to ensure they contribute equally to the algorithms like K-Means. The final df_final_features DataFrame is now shown and ready for clustering. [df_final_noshows.csv](#)

2.3 Model Development (Unsupervised Learning)

Unsupervised machine learning model used in this project is the K-Means model. K-means is a clustering algorithm used to partition n observations into k clusters, where each observation belongs to the cluster with the nearest mean (centroid).

1. Prune irrelevant features from preprocessed data

The development process start with pruning irrelevant features from preprocessed data:

```
# Choosing appropriate feature for K-means machine learning from preprocessed data
# These are chosen to find clusters and the characteristics of those clusters

# Identify all neighbourhood columns
neighbourhood_cols = [col for col in df_final_features.columns if 'Neighbourhood_' in col]

# Create a new DataFrame without irrelevant columns for clustering
X = df_final_features.drop(columns=neighbourhood_cols)
X = X.drop(columns=['AppointmentDayOfWeek', 'cluster', 'cluster_label'], errors='ignore')
```

All columns that have 'Neighbourhood_' at the front and column named 'AppointmentDaysOfWeek' are pruned to reduce the dimensionality of the features that will be used for clustering. As a result, features that is left are 'Gender', 'Age', 'Scholarship', 'Hypertension', 'Diabetes', 'Alcoholism', 'Handcap', 'SMS_receieved', 'WaitingDays', 'ScheduledDayOfWeek', and 'MonthOfAppointment'.

Columns named 'cluster' and 'cluster_label' are dropped in case there are old columns named with those names from the previous run of the code.

2. Determining Optimal Number of Clusters (k)

Next, the Elbow Method and Silhouette Score are used to determine the optimal number of clusters for the dataset. For the Elbow Method, the range of k values tested is from 1 to 11, resulting in a total of 10 trials using a random state of 0. Meanwhile, the Silhouette Score uses a range of k values from 2 to 11, with a total of 9 trials, since the minimum number of clusters must be 2. The code snippet for both methods are as follows:

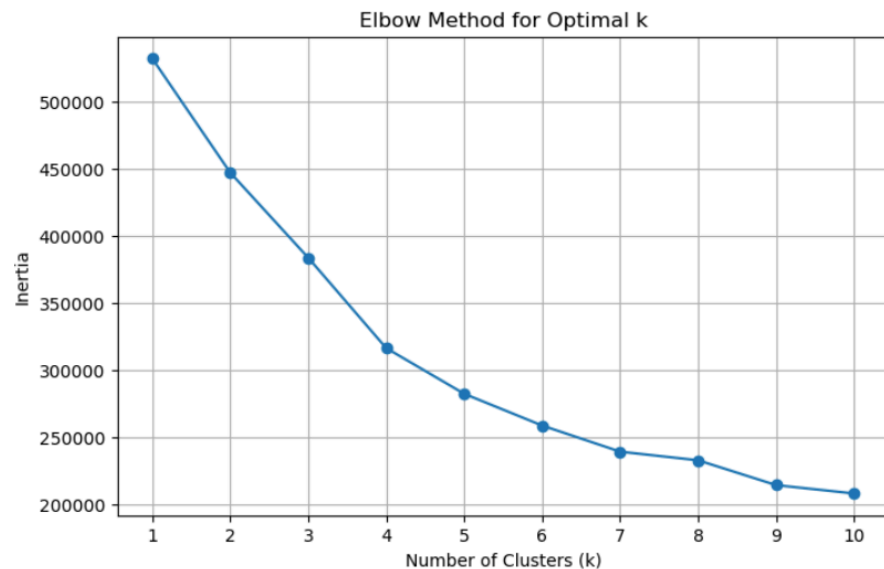
```
# Find Elbow point for k
inertia = []
for k in range(1, 11): # Test k from 1 to 10
    kmeans = KMeans(n_clusters=k, random_state=0, n_init='auto')
    kmeans.fit(X) # Use X (df_final_features) as your data input
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(range(1,11)) # Ensure x-axis ticks show all k values
plt.grid(True)
plt.show()
```

```
# Find Silhouette Scores for different k
silhouette_scores = []
for k in range(2, 11): # Silhouette score requires at least 2 clusters
    kmeans = KMeans(n_clusters=k, random_state=0, n_init='auto')
    cluster_labels = kmeans.fit_predict(X) # Use X (df_final_features)
    score = silhouette_score(X, cluster_labels)
    silhouette_scores.append(score)
    print(f"k={k}, Silhouette Score: {score:.3f}")

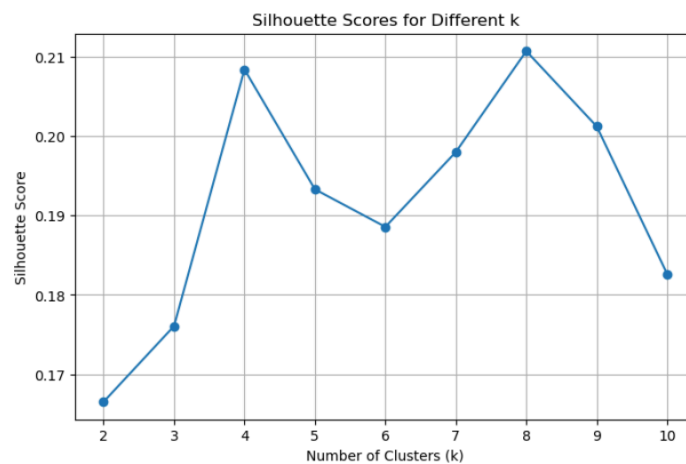
# plotting these scores for better visualization
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.title('Silhouette Scores for Different k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(range(2,11))
plt.grid(True)
plt.show()
```


The result of both methods are visualized using plot graphs. From the elbow method, the result shows that when $k=4$, is the optimal value of k . This is because as k increases, inertia always decreases but after a certain point, the rate of decrease slows down, this is where the “elbow” appears where $k=4$.



As observed from the silhouette score, $k = 8$ produced the best clustering quality (score = 0.211). However, $k = 3$ or 4 also produced reasonably strong scores (0.176 and 0.208) and were easier to interpret. Thus, for this project, we selected $k = 4$ based on a balance of mathematical quality and practical usefulness.

k=2, Silhouette Score: 0.167
k=3, Silhouette Score: 0.176
k=4, Silhouette Score: 0.208
k=5, Silhouette Score: 0.193
k=6, Silhouette Score: 0.189
k=7, Silhouette Score: 0.198
k=8, Silhouette Score: 0.211
k=9, Silhouette Score: 0.201
k=10, Silhouette Score: 0.183



After selecting the optimal number of clusters ($k = 4$), the K-Means algorithm was applied to the standardized dataset using the selected features. Each patient record was assigned to one of the four clusters based on their similarity across the selected variables.

The characteristics of each cluster were then analyzed by calculating the mean values of the features within each group. This helped to interpret what each cluster represents in terms of patient profiles. The clusters revealed distinct groups of patients, for example:

- Cluster 0: Middle-aged patients with moderate health risks but long waiting times
- Cluster 1: Younger individuals with no chronic conditions and short waiting times
- Cluster 2: Elderly patients with multiple health issues such as hypertension and diabetes
- Cluster 3: Patients with scholarships and a higher likelihood of receiving SMS reminders

Silhouette Score for this clustering is: 0.274

	Gender	Age	Scholarship	Hipertension	Diabetes	Alcoholism	Handcap	SMS_received	WaitingDays	ScheduledDayOfWeek	MonthOfAppointment	cluster
cluster												
0	0.31	0.09	0.08	0.15	0.05	0.02	0.01	0.62	2.20	0.05	-0.27	0.0
1	0.39	-0.82	0.13	0.01	0.00	0.01	0.01	0.24	-0.28	-0.06	-0.52	1.0
2	0.32	0.90	0.07	0.42	0.16	0.06	0.03	0.25	-0.32	0.06	-0.52	2.0
3	0.35	0.01	0.10	0.20	0.07	0.03	0.02	0.40	-0.16	-0.02	1.67	3.0

These clusters provide meaningful groupings that can be used to better understand patient behaviors and risk patterns. For example, patients in high-risk clusters (Cluster 2) may benefit from targeted reminders or improved appointment scheduling to reduce the likelihood of missed appointments.

2.4 Model Performance & Evaluation (Elbow Method, Silhouette Score)

Since K-Means is an unsupervised algorithm, traditional classification evaluation metrics such as accuracy, precision, recall and F1-score are not applicable. Instead, clustering performance is evaluated using internal validation methods that assess the quality of the clusters based on how cohesive and distinctive the clusters are with each other. The following are the methods used:

1. Silhouette Score

The silhouette score measures how well each data point fits within its cluster compared to other clusters. It ranges from -1 to 1, where higher values indicate better-defined clusters. In this project, the silhouette score was calculated for each value of k from 2 to 10.

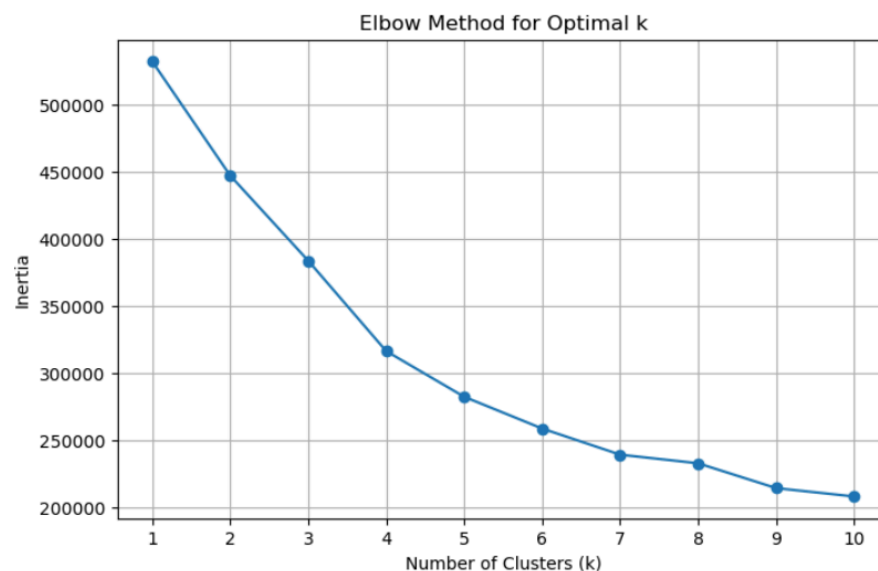
The highest silhouette score was observed at $k = 8$ (0.211), but $k = 4$ was selected due to better interpretability and sufficiently high score (0.208). However, after calculating the silhouette score again with a constant value of $k=4$ after cleaning the dataset again, the silhouette score results with a value of 0.274 which is an increase from 0.208.

Silhouette Score for this clustering is: 0.274

2. Elbow Method (Inertia)

Inertia is the sum of squared distances between each point and its assigned cluster center. A lower inertia indicates tighter clusters. However, since inertia always decreases as k increases, the elbow method helps identify the point where the rate of improvement sharply declines.

The elbow was observed at $k = 4$, indicating a good trade-off between model complexity and cluster cohesion.



3.0 CONCLUSION

This project successfully applied unsupervised machine learning techniques, specifically K-Means clustering, to uncover patterns in patient behavior regarding medical appointment attendance. After comprehensive data preparation and feature engineering, the Elbow Method and Silhouette Score were used to determine the optimal number of clusters. Although the silhouette score peaked at $k = 8$, $k = 4$ was selected based on a balance of interpretability, clustering quality, and practical usefulness.

Four different patient groups were identified by the final model, which had a silhouette score of 0.274 and was based on scheduling behavior, SMS reminders, health conditions, and demographics. These clusters can help healthcare providers identify high-risk patient profiles, such as elderly individuals with chronic conditions or patients with long waiting periods, who may require targeted actions.

By understanding the characteristics of each cluster, healthcare facilities can develop more effective appointment scheduling strategies and communication approaches to reduce no-show rates. This not only improves resource planning and efficiency but also enhances patient care delivery.

To further enhance clustering quality and actionability, future research could examine the use of supervised machine learning models to explicitly predict no-shows or take into account extra behavioral and social characteristics.